Assignment 21.5

Name: Navya Revuri

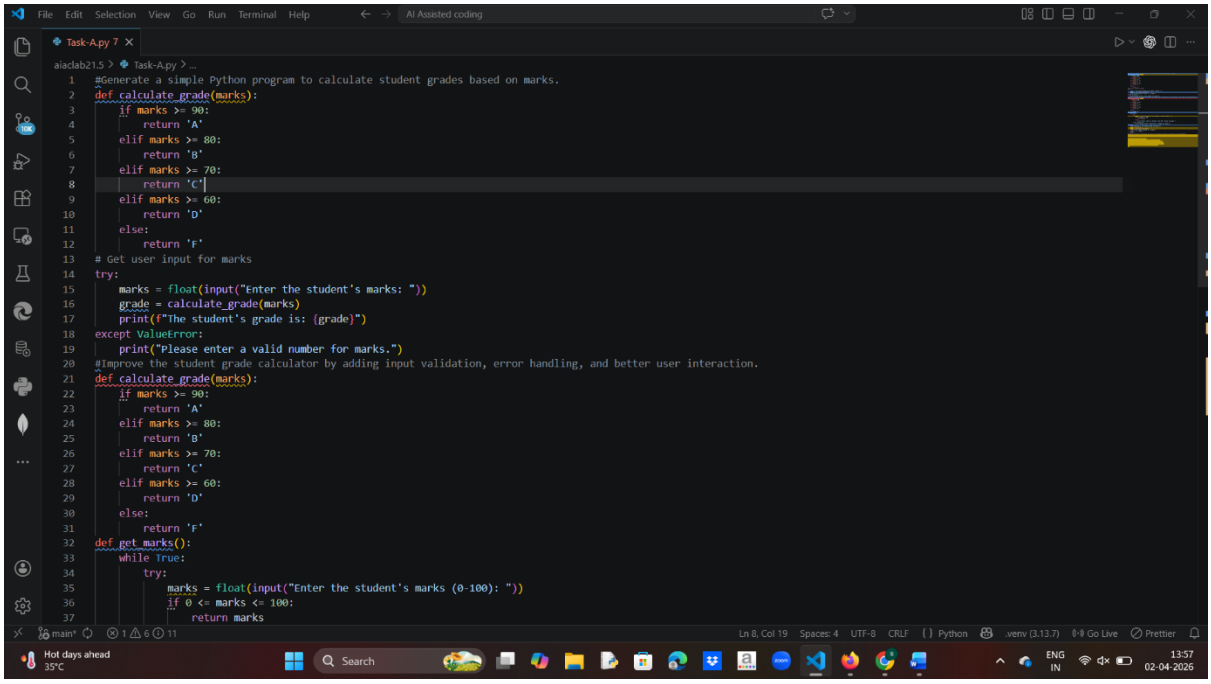
HTNO: 2303A52483

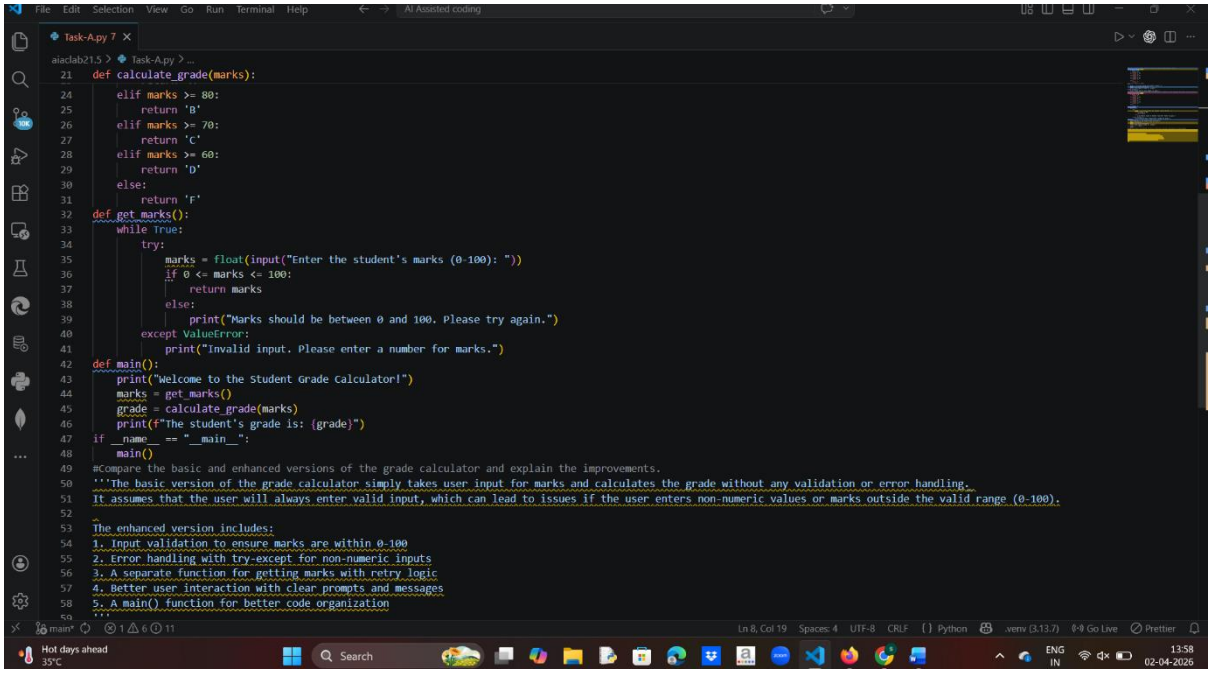
Batch 44

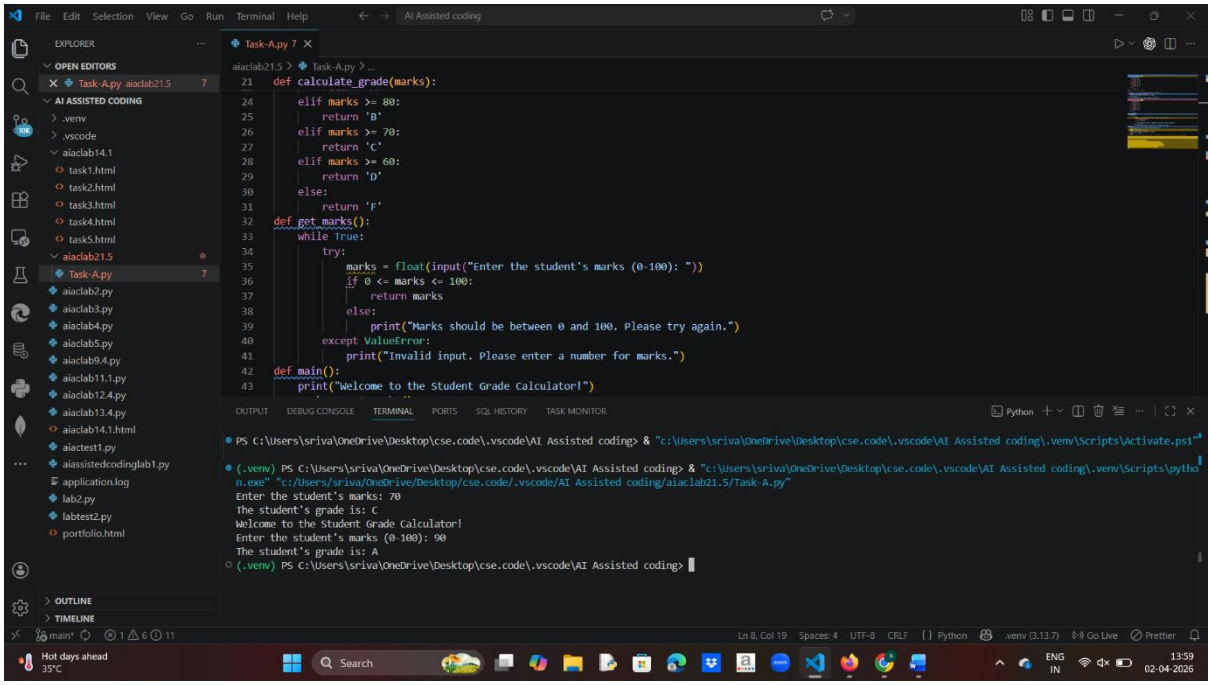
Task A – AI-Based Initial Program Development

Prompt:

Generate a Python program for a student grade calculator.
Take marks as input and calculate total and average.
Display grade based on average marks.
Then enhance the program with input validation.
Add features like multiple students and error handling







Observation

The initial program performs basic calculations only.
It does not check for invalid inputs like negative marks.
The enhanced version includes validation for correct data entry.
It supports multiple students, making it more practical.
Error handling improves reliability of the program.
Code structure becomes more modular and readable.

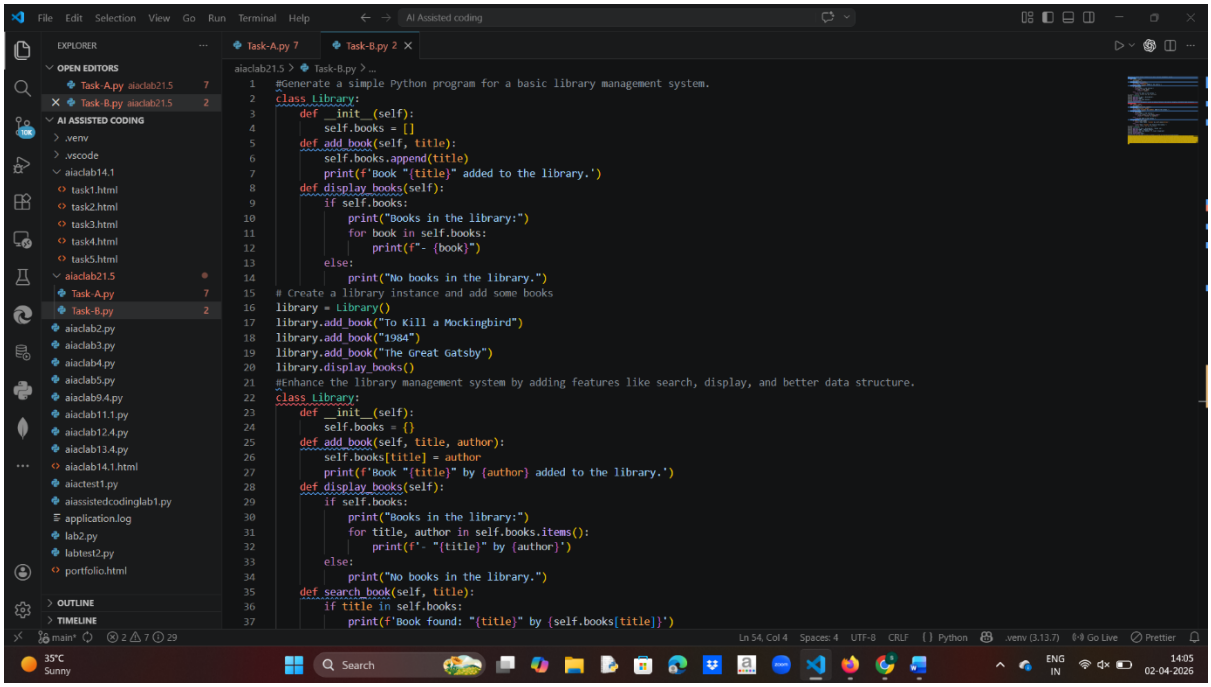
User experience is improved with proper prompts and messages.
Overall, AI significantly improves functionality and robustness**.**

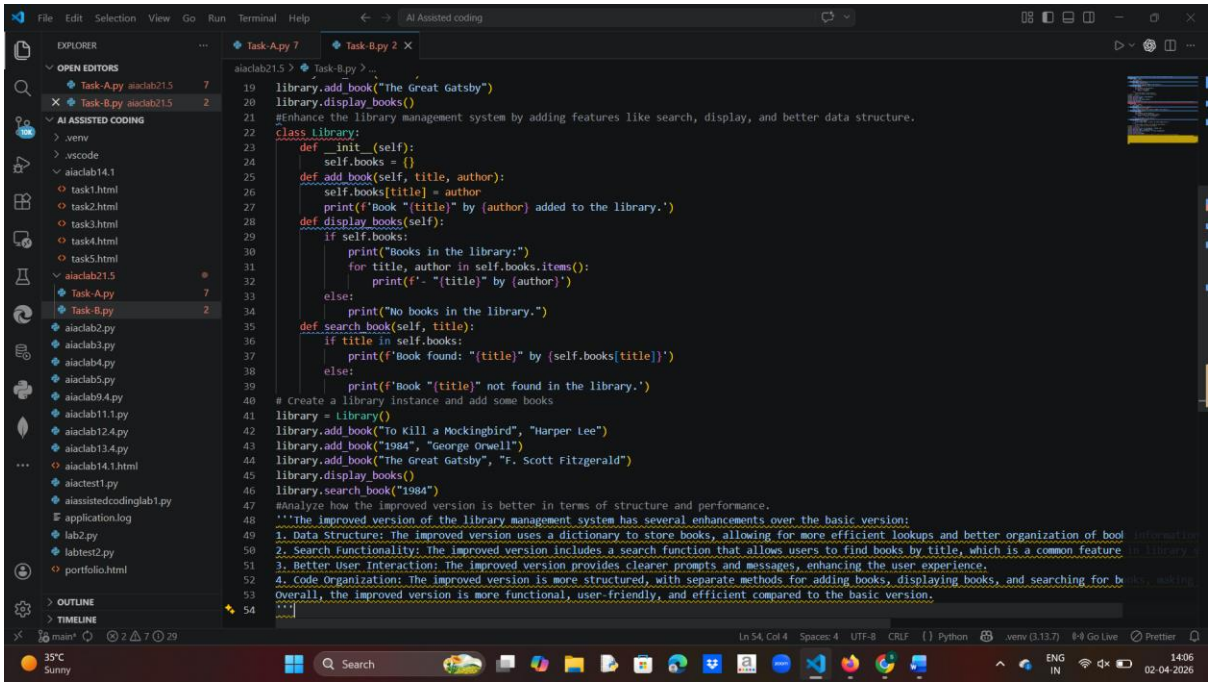
Task B – AI-Assisted Application Improvement

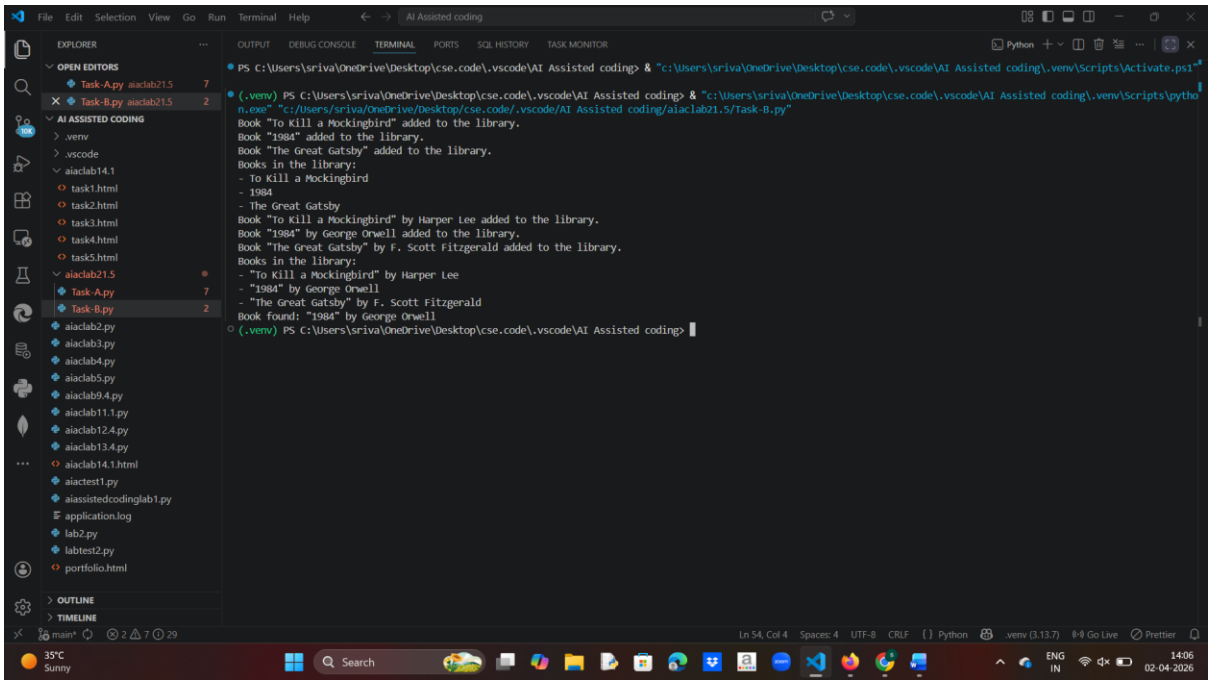
Prompt:

Generate a simple Python library management system.
Include features like adding and viewing books.
Then enhance it using AI by adding search functionality.
Include options for issuing and returning books.
Improve code structure and efficiency.

.







Observation:

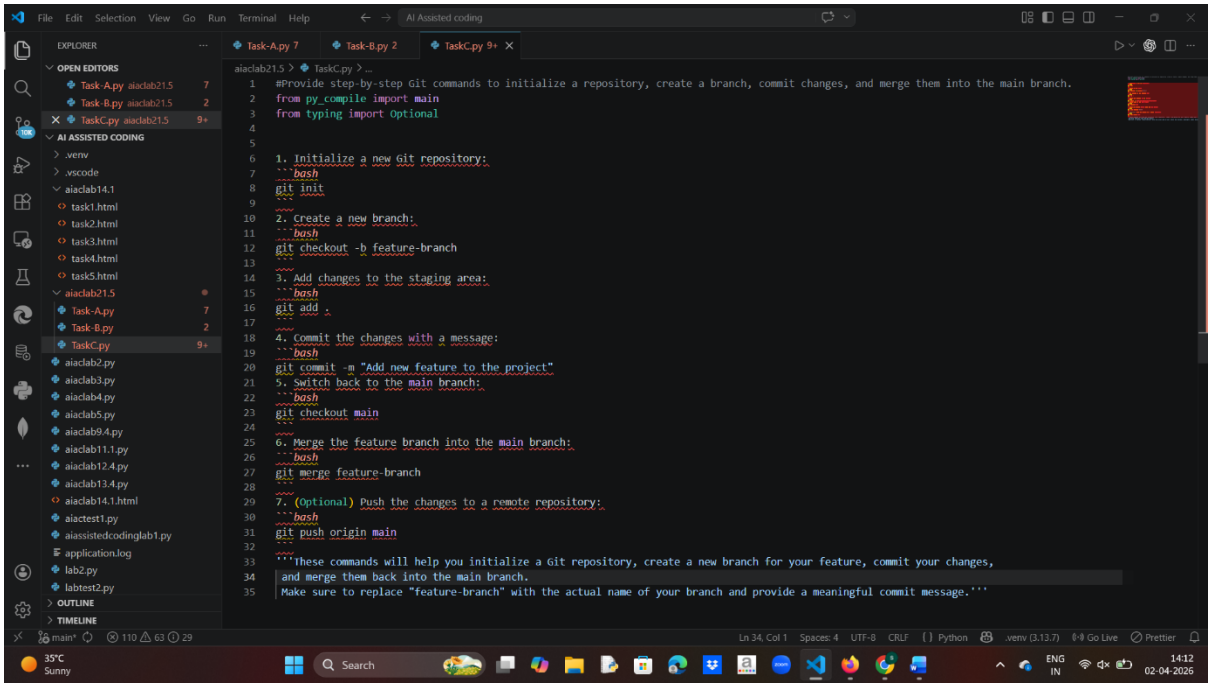
The initial program has limited features and basic structure.
It only supports adding and displaying books.
The improved version includes search and book issue features.
Function-based structure improves code organization.
Efficiency increases with better data handling.
User interaction becomes more dynamic and useful.

The program becomes closer to a real-world application.
AI helps transform a simple script into a complete system.

Task C – Git Workflow Practice

Prompt:

Initialize a Git repository for a Python project.
Create a new branch for adding new features.
Add AI-generated code changes in the branch.
Commit the changes with a message.
Merge the branch into the main branch.



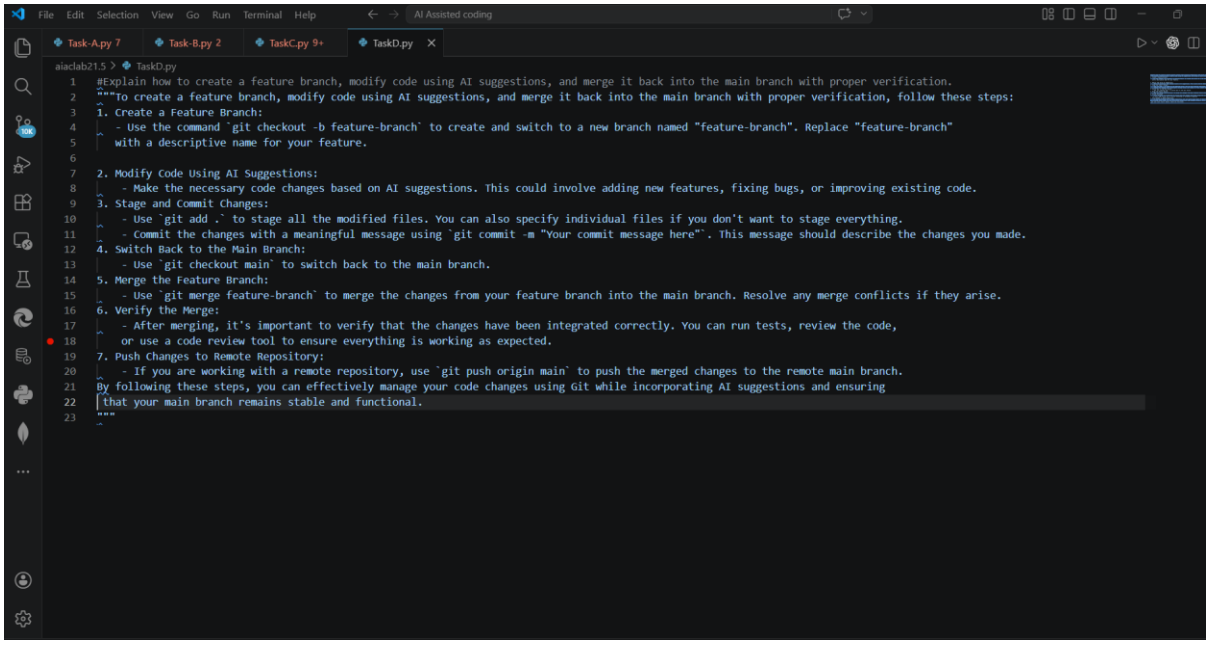
Observation:

Git initialization sets up version control for the project.
Branching allows safe development without affecting main code.
AI-generated changes can be tested separately in branches.
Commits help track changes with proper history.
Merging integrates new features into the main branch.
Conflicts may occur and require resolution.
Workflow improves collaboration and code management.
Using Git ensures better organization and rollback capability.

Task D – Branching and Merging with AI Support

Prompt:

Develop a basic program in the main branch.
Create a feature branch for modifications.
Use AI to enhance the program in the feature branch.
Merge the updated branch into main.
Verify the final program output.



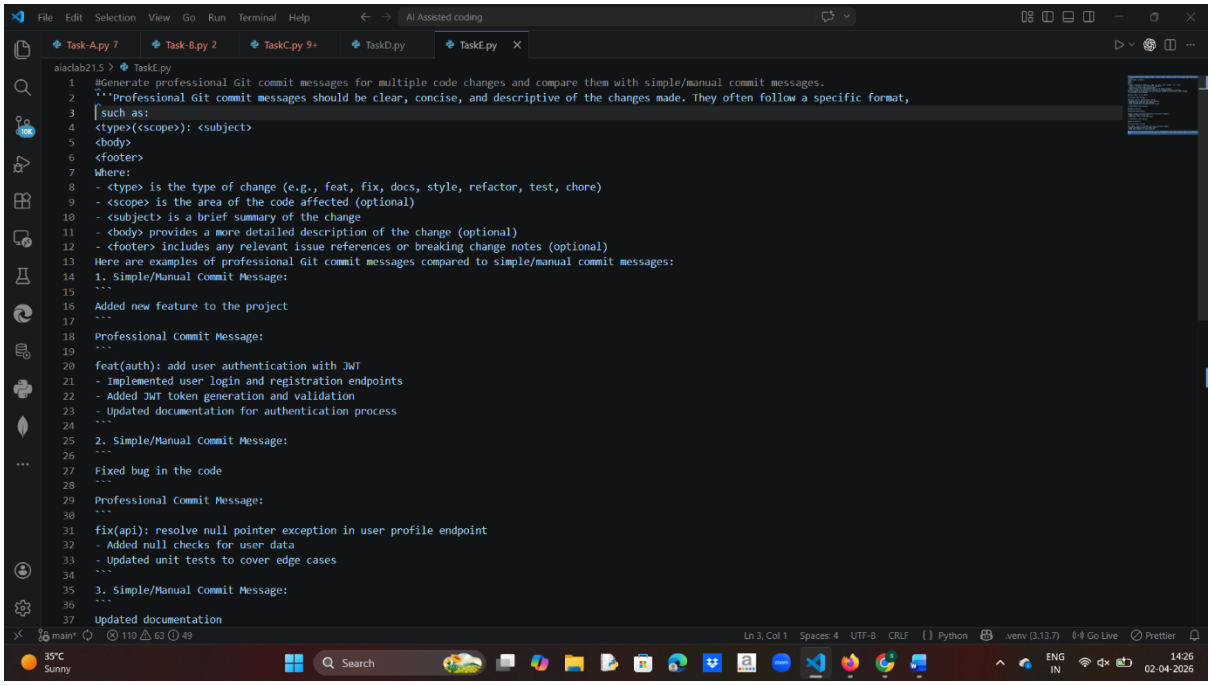
Observation

The main branch contains the stable version of the program.
Feature branch allows safe experimentation and improvements.
AI helps add new features quickly in the branch.
Merging combines both versions into a single codebase.
Testing ensures that merged code works correctly.
Conflicts can occur if both branches modify same lines.
Proper merging maintains code integrity.
This workflow supports structured and safe development.

Task E – AI for Commit Message Writing

Prompt:

Make multiple changes to a Python program.
Write manual commit messages for each change.
Then use AI to generate commit messages.
Compare both manual and AI-generated messages.
Analyze clarity and usefulness.



Observation:

Manual commit messages may be unclear or inconsistent.
AI-generated messages are usually more structured and descriptive.
AI highlights key changes effectively in fewer words.
Consistency is better in AI-generated messages.
Manual messages depend on developer experience.
AI helps beginners write professional commit messages.
However, AI may sometimes lack context of deeper changes.
Combining manual understanding with AI gives best results.